Cours 2 : Réseaux de neurones « simples » NN, CNN, RNN

université de BORDEAUX

Before starting

- **Contact:** victoria.bourgeais@u-bordeaux.fr
- **Schedule:** 12 weeks (37-43,45-49), 48h CI
  - 10/09 (4h) : Outillage
  - **19/09 (4h) : Réseaux de neurones « simples » : NN, CNN, RNN**
  - 24/09 : annulé
  - 01/10 (4h) + 08/10 (2h) : ResNet, VGG, U-Net
  - 08/10 (2h) + 15/10 (4h) : Modèles génératifs : Autogénération, GAN, Auto-encodeurs, VAE, (Akka)
  - 22/10 (4h) : Architectures pour le NLP : LSTM, Embeddings, NPLM, Mécanisme d'attention, tranformers à GPT
  - 05/11 (4h) : Modèles de graphes : GNN, GCNN
  - 12/11 (4h) : Réseaux pour les données multimodales : CLIP, …
  - 19/11 (4h) :Stable diffusion
  - 26/11 et 03/12 (12h) : Projet

Before starting

- Working remotely: https://services.emi.u-bordeaux.fr/intranet/spip.php?article175
- Set the virtual environnement as a Jupyter kernel:
  1. Open a terminal

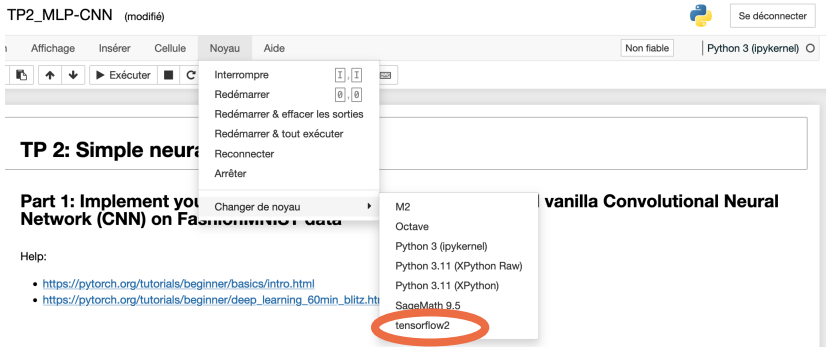
  2. Activate the virtual environment:

```
source espaces/ens/DeepLearning/python3/arna/bin/activate
```

  3. Run the following command (supposed ipykernel is installed)

```
python -m ipykernel install --user --name=tensorflow2
```

  4. When launching a jupyter notebook, you should see it:



Major AI Conferences and Events

Major AI Conferences and Events

Area	Major venues	Main focus
General Artificial Intelligence	AAAI, IJCAI, ECAI	Broad AI research: reasoning, planning, knowledge representation, learning and applications
Machine Learning and Deep Learning	NeurIPS, ICML, ICLR	Machine learning, deep learning, optimization and representation learning
Machine Learning and Data Mining	ECML PKDD, ACM SIGKDD	Machine learning, data mining, knowledge discovery and applied data science
Responsible AI and Society	AAAI/ACM AIES, ACM FAccT	Ethics, fairness, accountability, transparency and societal impacts

Major AI Conferences and Events

Industry, Innovation and Policy Events

Event	Orientation
VivaTech	Technology, startups, industry and innovation
Big Data & AI Paris	Enterprise data and applied AI
RAISE Summit	AI industry, investment, startups and public policy
AI for Good Global Summit	Societal applications, standards and international cooperation, led by the ITU/UN
AI Action Summit — Paris 2025	International and governmental AI policy summit

Overview

1. Neural network: ingredients
   Multi-Layer Perceptron (MLP)

2. Convolutional Neural Network (CNN)
   A simple vanilla CNN: LeNet

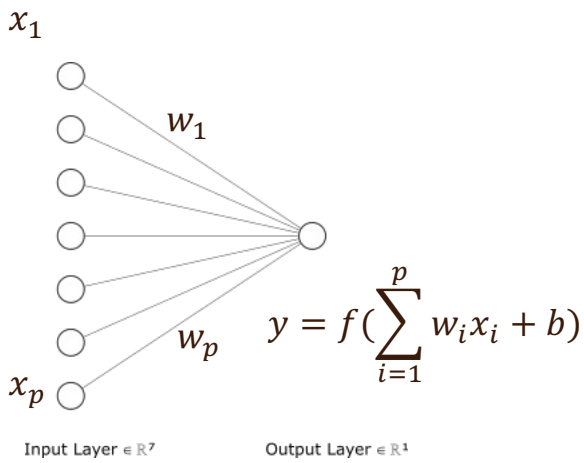
Overview

1. **Neural network: ingredients**
   Multi-Layer Perceptron (MLP)

2. Convolutional Neural Network (CNN)
   A simple vanilla CNN: LeNet

From perceptron to multi-perceptron

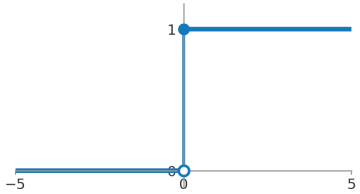
(Rosenblatt, 1957)



From perceptron to multi-perceptron

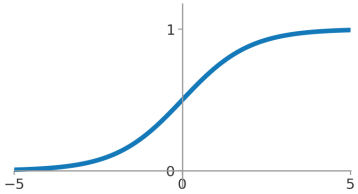
Heaviside

$H(x) = 0,\ x < 0$
$H(x) = 1,\ x \geq 0$



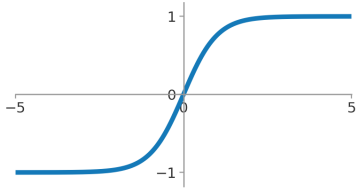
Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



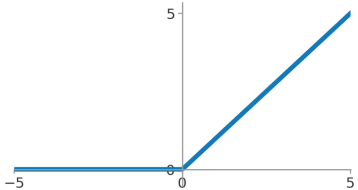
tanh

$\tanh(x)$



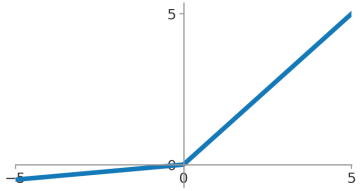
ReLU

$\max(0, x)$



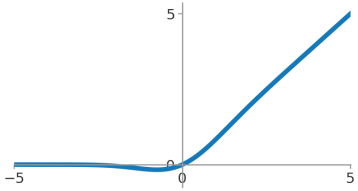
Leaky ReLU

$\max(0.1x, x)$



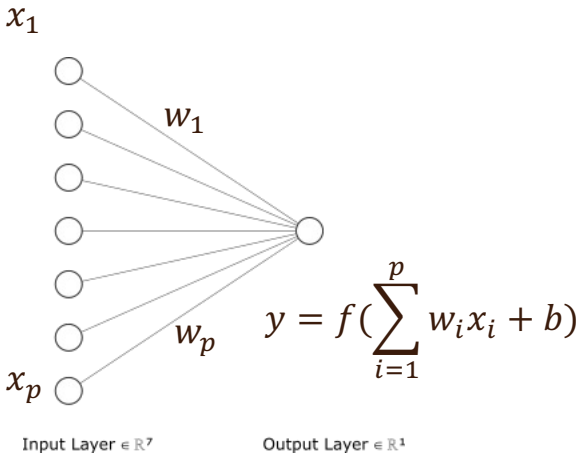
GELU

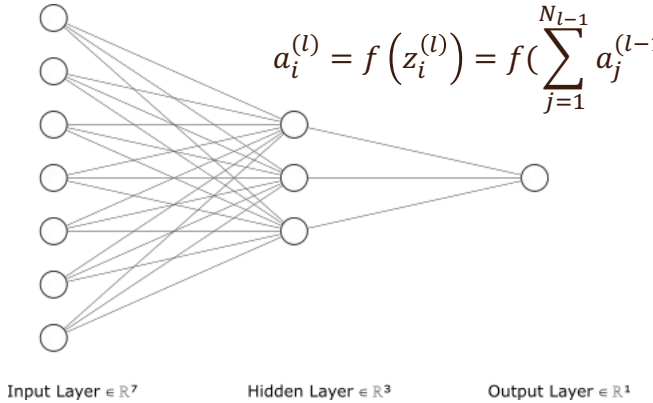
$x\,\Phi(x)$



From perceptron to multi-perceptron

(Bengio et al., 2017)



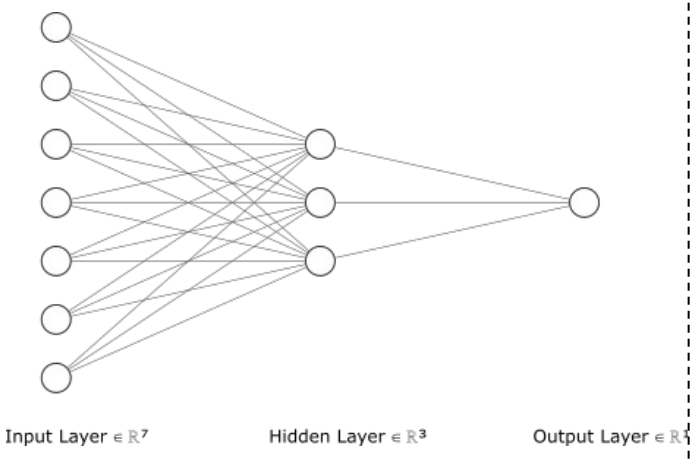


The set of parameters of a MLP with L layers is defined as:

$$\theta = \{(W^{(1)}, b^{(1)}), (W^{(2)}, b^{(2)}), \dots, (W^{(L)}, b^{(L)})\}$$

$W^{(l)}$ has a shape of (n_l, n_{l-1}) and $b^{(l)}$ $(n_l, 1)$

From perceptron to multi-perceptron



Implementation in PyTorch

```
import torch.nn as nn

class VanillaMLP(nn.Module):
    def __init__(self,n_classes):
        super(VanillaMLP,
        self).__init__()
        self.fc1 = nn.Linear(7, 3)
        self.fc2 = nn.Linear(3,
        n_classes)
        self.relu = nn.ReLU()

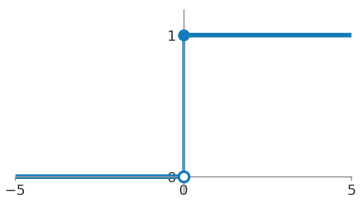
    def forward(self, x):
        h1 = self.relu(self.fc1(x))
        y = self.fc2(h1)
    return y
```

A MLP can be implemented with sklearn: https://scikit-learn.org/stable/modules/neural_networks_supervised.html

Output layer: activation function and loss

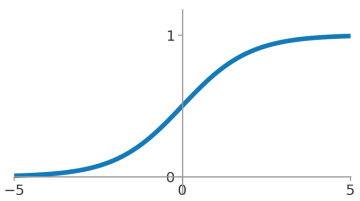
Heaviside

$H(x) = 0,\ x < 0$
$H(x) = 1,\ x \geq 0$



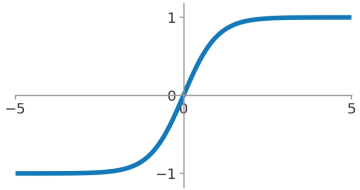
Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



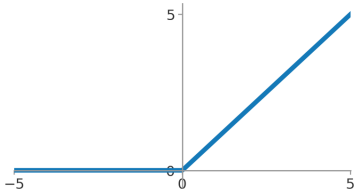
tanh

$\tanh(x)$



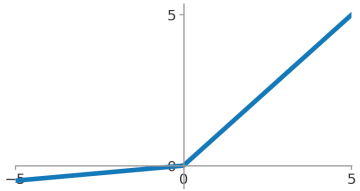
ReLU

$\max(0, x)$



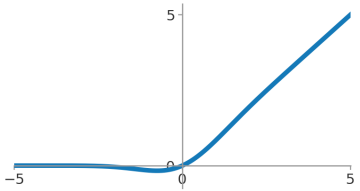
Leaky ReLU

$\max(0.1x, x)$



GELU

$x\,\Phi(x)$

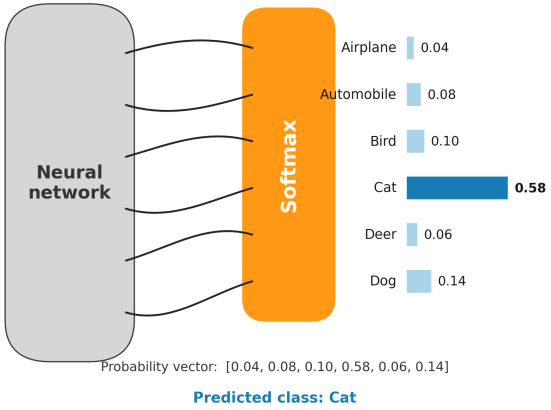


Output layer: activation and loss functions

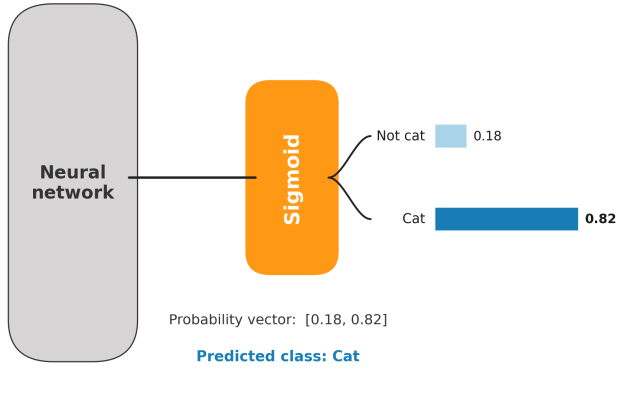
- It depends on the type of learning: supervised, unsupervised or reinforcement
- In case of supervised learning: two cases

  ➢ Classification: compute a probability

Multiclass classification



Binary classification



Output layer: activation and loss functions

- It depends on the type of learning: supervised, unsupervised or reinforcement
- In case of supervised learning: two cases

➢ Classification: compute a probability

- Binary:

Use of sigmoid function such as $y = a^{(L)} = \frac{1}{1-\exp(z^{(L)})}$

Use of Binary Cross-Entropy (BCE) Loss: $\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$

- Multiclass:

Use of softmax function such as $y_c = a_c^{(L)} = \frac{\exp(z_c^{(L)})}{\sum_{j=1}^{C}\exp(z_j^{(L)})}$ with C the number of classes

→ one output neuron by class c

Use of Cross-Entropy (CE) Loss: $\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c})$

Recommended when #C>2.

Output layer: activation and loss functions

- It depends on the type of learning: supervised, unsupervised or reinforcement
- In case of supervised learning: two cases

  - Classification: compute a probability
    - Binary: in practice

```
criterion = nn.BCEWithLogitsLoss()
output_fn = nn.Sigmoid()
```

    - Multiclass: in practice

```
criterion = nn.CrossEntropyLoss()
output_fn = nn.Softmax(dim=1)
```

Output layer: activation and loss functions

- It depends on the type of learning: supervised, unsupervised or reinforcement
- In case of supervised learning: two cases

  ➢ Regression :

  Use of identity function such as $y = a^{(L)} = z^{(L)}$

  Use of Mean-square error (MSE) Loss: $\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$

  In practice :

```
criterion = nn.MSELoss()
```

Gradient-descent optimization

- To minimize the loss and optimize the parameters of your NN: you should use a gradient descent algorithm.

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \widehat{\mathbf{g}}_t$$

Where: $$\widehat{\mathbf{g}}_t = \nabla_{\boldsymbol{\theta}} \mathcal{J}_{B_t}(\boldsymbol{\theta}_t) = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\boldsymbol{\theta}} \ell\left(f_{\boldsymbol{\theta}_t}(x_i), y_i\right)$$

$\boldsymbol{\theta}_t$: model parameters at training step t,
B_t : mini-batch used at training step t,
ℓ : loss for one observation,
$\mathcal{J}_{B_t}$: average loss over mini-batch B_t,
$\widehat{\mathbf{g}}_t$: mini-batch gradient computed by backpropagation,
η_t : learning rate at training step t.

Gradient-descent optimization: two optimizer families to know

SGD+momentum

One global learning rate
Momentum smooths successive gradients and accelerates movement along persistent directions.

Strengths
Simple, memory-efficient and a strong baseline, especially for many CNN and vision settings.

Main sensitivity
The learning rate and its schedule require careful tuning.

AdamW

Adaptive step for each parameter
Adam tracks the first and second moments of gradients. AdamW adds decoupled weight decay.

Strengths
A common default for transformers and many modern architectures. It often converges quickly.

Main cost
Additional optimizer states increase memory use.

Gradient-descent optimization: in practice

- There exist different ones: SGD, SGD with mini-batch, Adam… Most of them are implemented in PyTorch in the module `torch.optim`

```
import torch.optim as optim

optimizer = optim.SGD(net.parameters(), lr=0.001)
for epoch in np.arange(N_EPOCHS):
    for idx,batch in enumerate(trainloader): #training loop
    # get the inputs; batch is a list of [inputs, labels]
        inputs,labels=batch
        inputs=inputs.to(device) #train on GPU
        labels=labels.to(device)
        # zero the parameter gradients
        optimizer.zero_grad()
        # forward + backward + optimize
        out = net(x=inputs)
        loss = criterion(out, labels)
        loss.backward()
        optimizer.step()
```

Gradient-descent optimization: learning rate scheduler



Gradient descent

- There exist different ones: SGD, SGD with mini-batch, Adam… Most of them are implemented in PyTorch in the module [torch.optim]

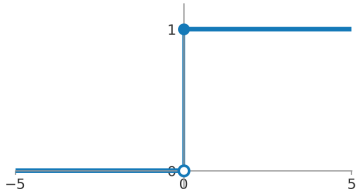
```
import torch.optim as optim

optimizer = optim.SGD(net.parameters(), lr=0.001)
for epoch in np.arange(N_EPOCHS):
     for idx,batch in enumerate(trainloader): #training loop
          inputs,labels=batch
          inputs=inputs.to(device) #train on GPU
          labels=labels.to(device)
          optimizer.zero_grad()
          scheduler =
          torch.optim.lr_scheduler.CosineAnnealingLR(
              optimizer, T_max=n_epochs)
          out = net(x=inputs)
          loss = criterion(out, labels)
          loss.backward()
          optimizer.step()
          Scheduler.step()
```

Activation functions

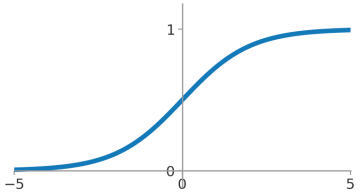
Heaviside

$H(x) = 0,\ x < 0$
$H(x) = 1,\ x \geq 0$



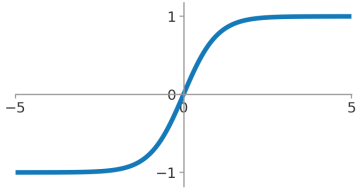
Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



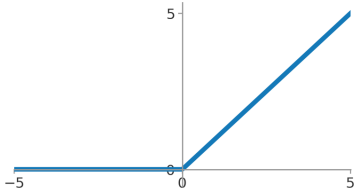
tanh

$\tanh(x)$



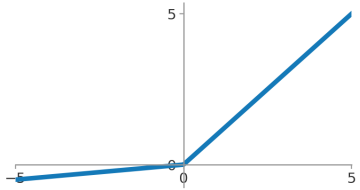
ReLU

$\max(0, x)$



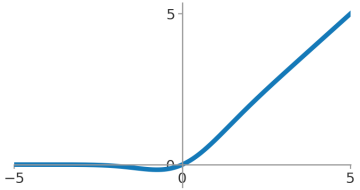
Leaky ReLU

$\max(0.1x, x)$



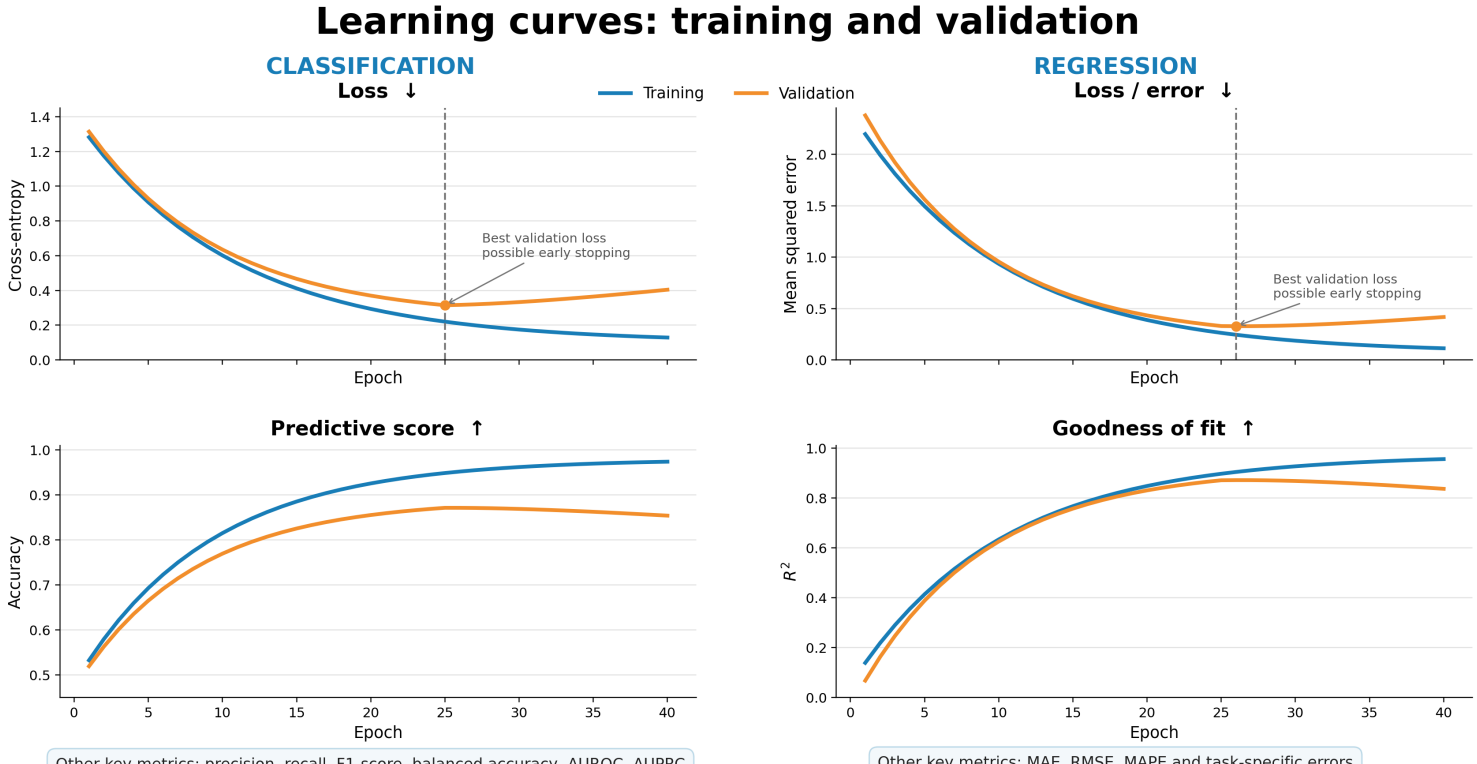
GELU

$x\,\Phi(x)$



How to evaluate?

Functions are available in:
https://lightning.ai/docs/torchmetrics/stable



Illustrative curves — values do not come from a real experiment.

How to evaluate? Use of PytorchMetrics

```
import torch
from torchmetrics.regression import R2Score

r2_metric = R2Score()

model.eval()

with torch.no_grad():
    for x, y in test_loader:
        predictions = model(x).squeeze()

        r2_metric.update(predictions, y)

r2 = r2_metric.compute()
print(f"R² = {r2.item():.3f}")

r2_metric.reset()
```

```
metric = R2Score()
r2 = metric(y_pred, y_true)
```

Functional API

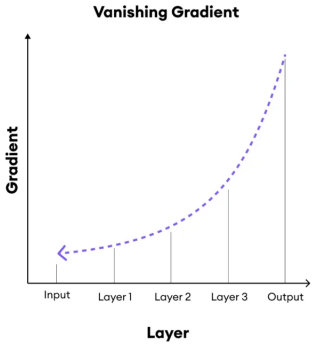
```
from torchmetrics.functional.regression import
r2_score

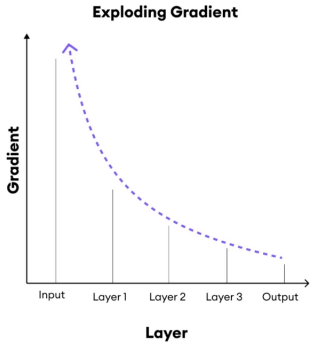
r2 = r2_score(y_pred, y_true)
```

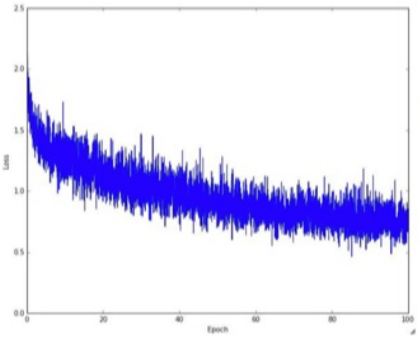
Assignment 1

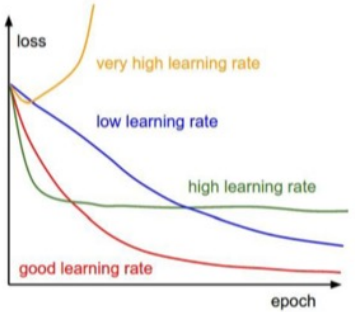
Diagnose learning problems (most common ones)

- **Vanishing Gradient**: gradients tend to nullify (in particular, fully-connected and large neural networks are sensitive).
- **Exploding Gradient**: gradients become very large.



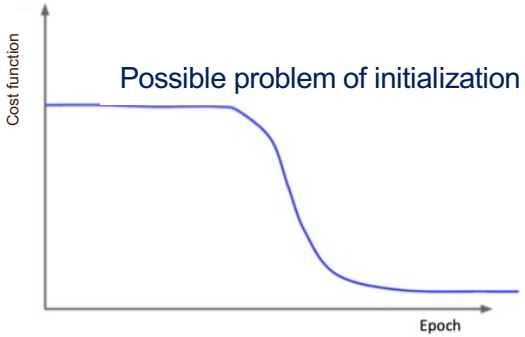


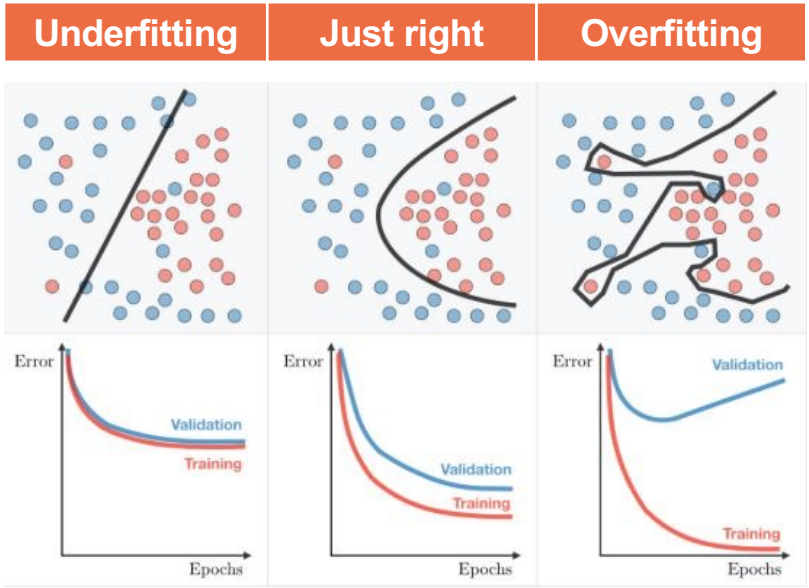




Diagnose learning problems (most common ones)

- Overfitting/underfitting
- Problem of initialization





➔Complexify the model
➔Add more features

➔Perform regularization

Which solutions?

- Appropriate initialization of the model parameters
- Regularization techniques
  - Early stopping
  - L1,L2 Regularization
  - Batch Normalization
  - Dropout

Initialization of model parameters

- Initialization will affect the speed of convergence of the gradient descent algorithm.
- Initialization with zero weights doesn't work most of the time.
- In Pytorch, the parameters are initiated with a uniform distribution: $U(-\sqrt{k}, \sqrt{k})$ where $k = \frac{1}{fan_in}$ (check the documentation of `nn.Linear`).
- The choice of initialization is linked to the activation function (have a look at `nn.init`) with `fan_in`=number of input neurons and `fan_out`=number of input neurons.

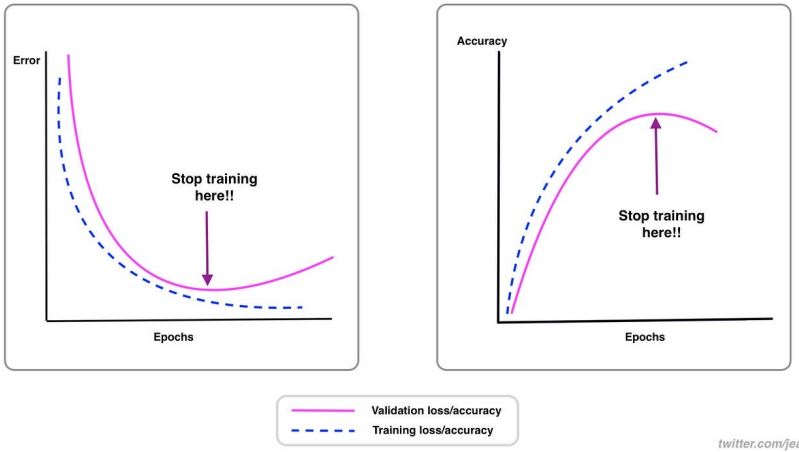
Initialization	Activation functions	σ (normal distribution)
Glorot	Identity, tanh, logistic, softmax	$\frac{1}{\sqrt{fan_{in} + fan_{out}}}$
He	ReLU and variants	$\sqrt{\frac{2}{fan_{in}}}$

Regularization techniques

To implement manually

There is a class that exists in PyL

- **Early stopping:**
  - Check the learning curves over time.
  - Stop the training when the model starts to **overfit** within a specific window, supposing using a validation set.



Regularization techniques

- **L1,L2 Regularization:** facilitate a parsimonious solution.

Lasso regularization (L1-loss): $\mathcal{L}_{total} = \mathcal{L}_{CE} + \frac{\lambda}{2N}\sum_{l=1}^{L}\left\|W^{(l)}\right\|_1$

where $\left\|W^{(l)}\right\|_1 = \sum_{i,j} w_{i,j}$

Ridge regularization (L2-loss): $\mathcal{L}_{total} = \mathcal{L}_{CE} + \frac{\lambda}{2N}\sum_{l=1}^{L}\left\|W^{(l)}\right\|_2^2$

where $\left\|W^{(l)}\right\|_2^2 = \sum_{i,j} w_{i,j}^2$

Regularization techniques

`nn.BatchNorm1d`

Some versions exist to work on top of convolutional layers

- **Batch normalization (BN):** reduce the problem of covariate shift, i.e. changes in the distribution of internal activations during training. By normalizing the pre-activations ($\{Z_l\}$) in each layer l, BN makes training more stable and enables higher learning rates to be used.

At a given layer l:

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\ldots m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

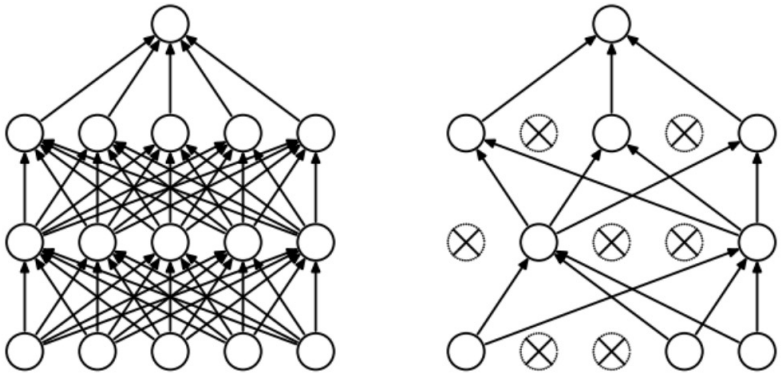
$$y_i \leftarrow \gamma\widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

where x_i represents here the pre-activation $z_{i,l}$ for sample i at layer l

During inference or test time, the mean and variance are fixed. They are estimated using the previously calculated means and variances of each training batch.

Regularization techniques

- **Dropout:** `nn.Dropout`
  - Set randomly the activation of some neurons to 0 with a `p` probability.
  - It forces the network to learn several solutions (improve the robustness).
  - Helpful to limit the problem of vanishing gradients.



(a) Standard Neural Net (b) After applying dropout.

Full pipeline:

```
nn.Linear
nn.BatchNorm1d
nn.ReLU
nn.Dropout
```

Good practice

- Don’t forget your data:
  - Preprocessing is important.
  - Problem of data imbalance and data size.
  - Some models work pretty well with some data: CNN for image data, RNN/Transformers for text data…
  - Assuming you have two datasets A and B of same type (i.e., image), it’s not because your model works well on A that it will work well on A.
  - Split your data in at least three sets: train, test, val. If you don’t have enough data, you can use cross-validation.

Good practice

- Track your training using TensorBoard, WeightsAndBiases, MLFlow…
- Hyper finetune your model using Optuna, Ray Tune…
- Comment your code useful either for yourself or your collaborators
- Start with a simple model and then custom the model

Assignment 1 – continue!

Overview

1. Neural network: ingredients
   Multi-Layer Perceptron (MLP)

2. **Convolutional Neural Network (CNN)**
   A simple vanilla CNN: LeNet

Convolutional Neural Network (CNN)

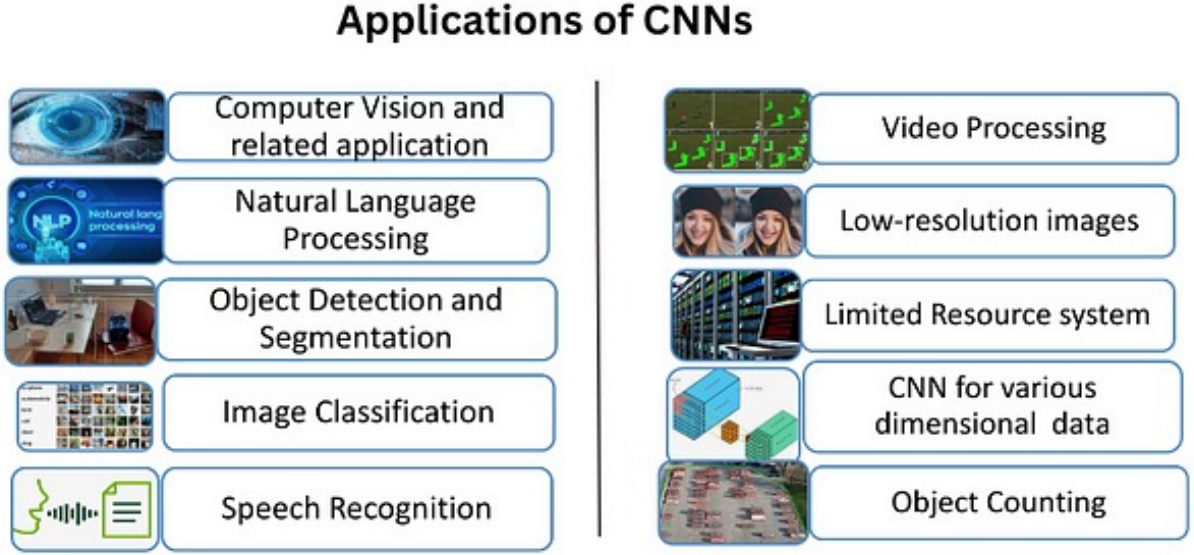
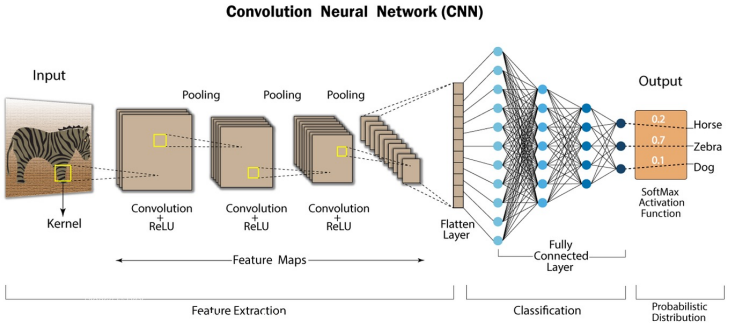
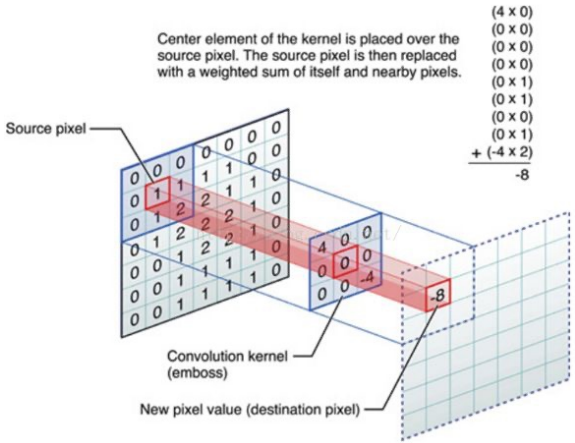


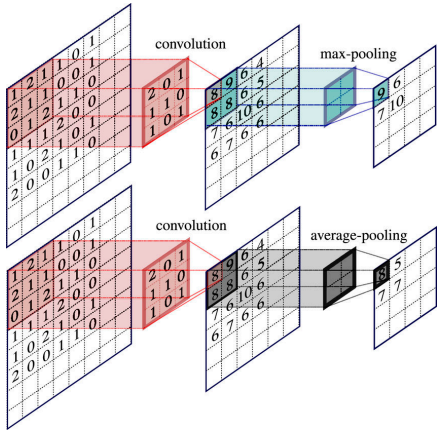
Image source: https://medium.com/@aiblogtech/convolutional-neural-network-a-simple-and-detailed-guide-bd0fbdc8cedb

CNN: main ingredients

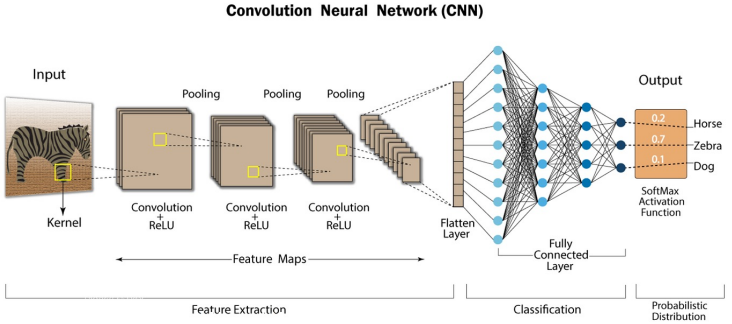


- Convolution layer: `nn.Conv2d`
- Pooling layer: `nn.AvgPool2d`, `nn.MaxPool2d`
- Dense layer = MLP

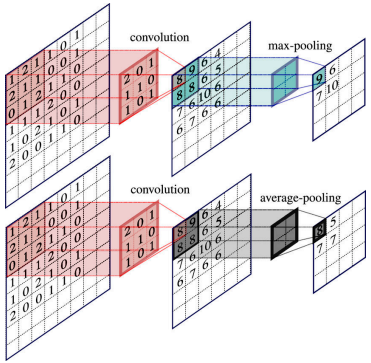




CNN: main ingredients



- Convolution layer: `nn.Conv2d`
- Pooling layer: `nn.AvgPool2d`, `nn.MaxPool2d`
- Dense layer = MLP



- Input: $(N, C_{in}, H_{in}, W_{in})$ or (C_{in}, H_{in}, W_{in})
- Output: $(N, C_{out}, H_{out}, W_{out})$ or $(C_{out}, H_{out}, W_{out})$, where

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times \text{padding}[0] - \text{dilation}[0] \times (\text{kernel_size}[0] - 1) - 1}{\text{stride}[0]} + 1 \right\rfloor$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times \text{padding}[1] - \text{dilation}[1] \times (\text{kernel_size}[1] - 1) - 1}{\text{stride}[1]} + 1 \right\rfloor$$

Variables

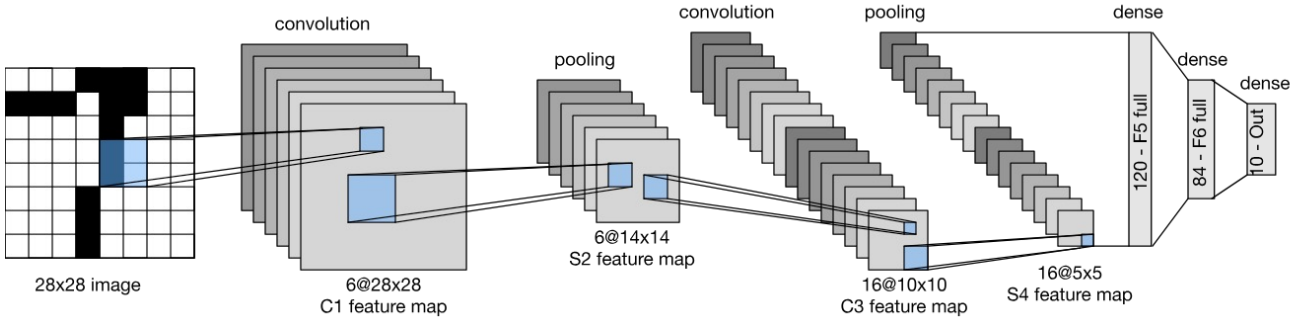
- **weight** (*Tensor*) – the learnable weights of the module of shape $(\text{out_channels}, \frac{\text{in_channels}}{\text{groups}}, \text{kernel_size}[0], \text{kernel_size}[1])$. The values of these weights are sampled from $\mathcal{U}(-\sqrt{k}, \sqrt{k})$ where $k = \frac{groups}{C_{\text{in}} * \prod_{i=0}^{1} \text{kernel_size}[i]}$
- **bias** (*Tensor*) – the learnable bias of the module of shape (out_channels). If `bias` is `True`, then the values of these weights are sampled from $\mathcal{U}(-\sqrt{k}, \sqrt{k})$ where $k = \frac{groups}{C_{\text{in}} * \prod_{i=0}^{1} \text{kernel_size}[i]}$

CNN: main ingredients

- Features extracted by cnn blocks along the architecture

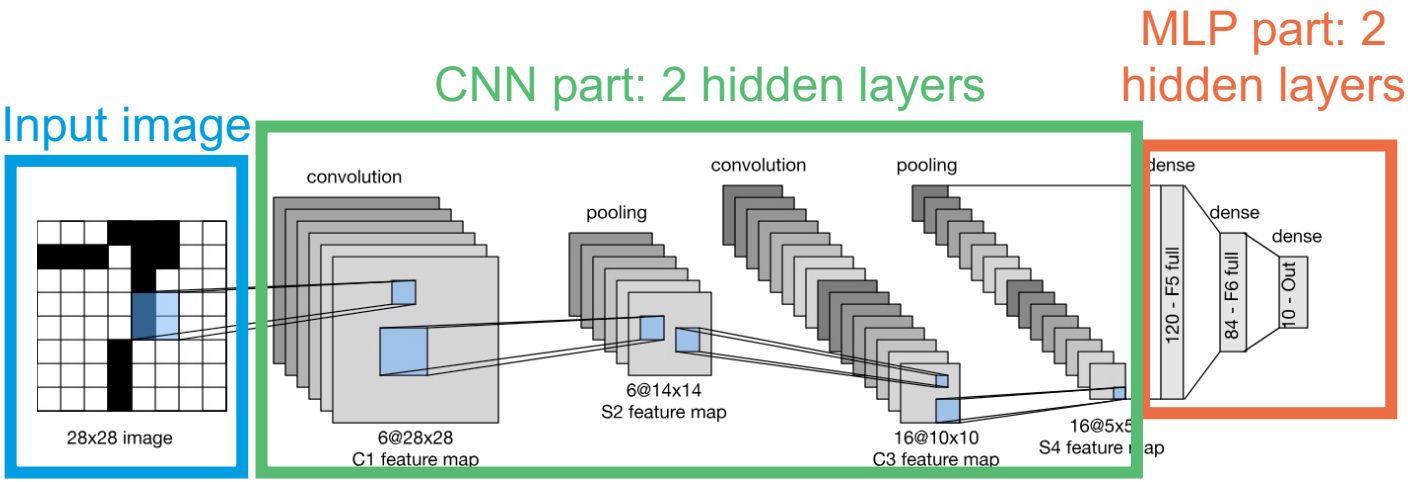


CNN: let’s get familiar with LeNet model



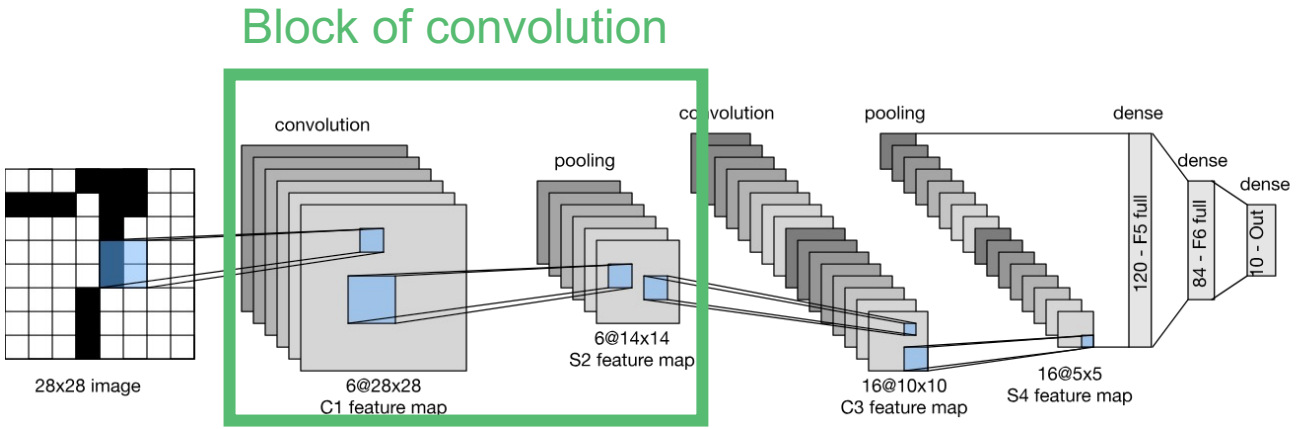
Introduced by Yann LeCun et al. in 1998, the first works date from 1989.

CNN: let’s get familiar with LeNet model

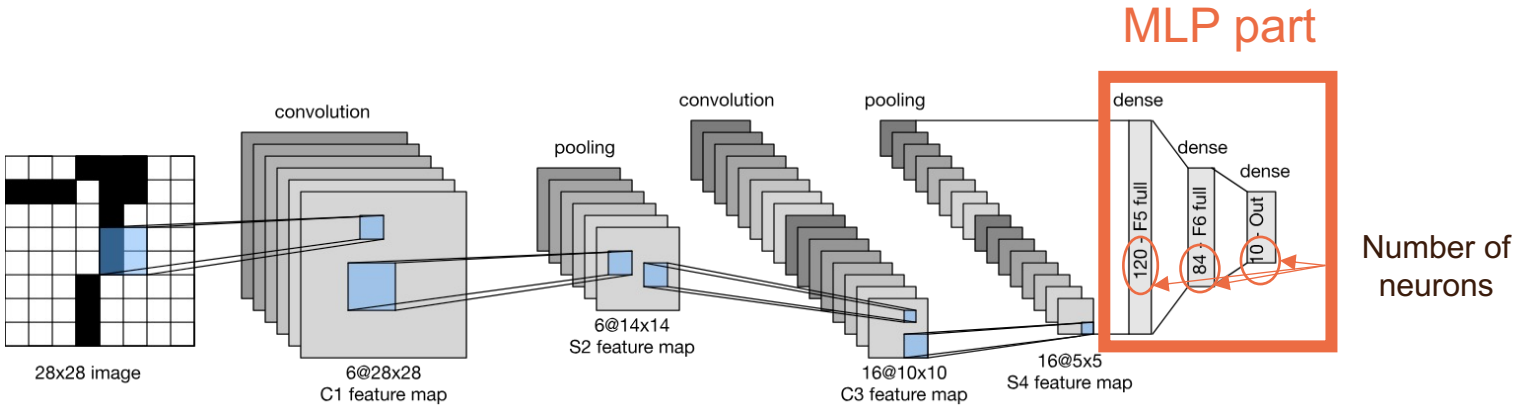


Introduced by Yann LeCun in 1998, the first works date from 1989.

CNN: let's get familiar with LeNet model



CNN: let's get familiar with LeNet model





Assignment 1 - continue

Overview

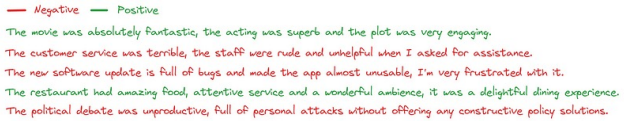
1. Neural network: ingredients
   Multi-Layer Perceptron (MLP)

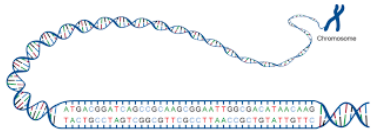
2. Convolutional Neural Network (CNN)
   A simple vanilla CNN: LeNet

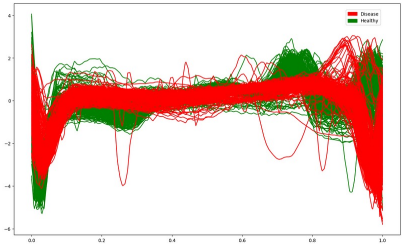
3. Recurrent Neural Network (RNN)

Deep Learning for sequential data

What for? Deal with sequential data







- A **sequence** is a observation $\{x^{<t>}\}$ for $t \in \{1; T\}$. Generally, the observation at the time step t depends on the previous observations.

  Some examples: text, DNA, time series…

➔ We need to find a model that considers this temporality and that predicts $y^{<t>}$ for each $x^{<t>}$.

Deep Learning for sequential data

➔ 1D-CNN can be used, but it cannot manage that:

- ➢ the sequences could be of different sizes (the window size is fixed)
- ➢ the predictions could be dependent (no memory effect)

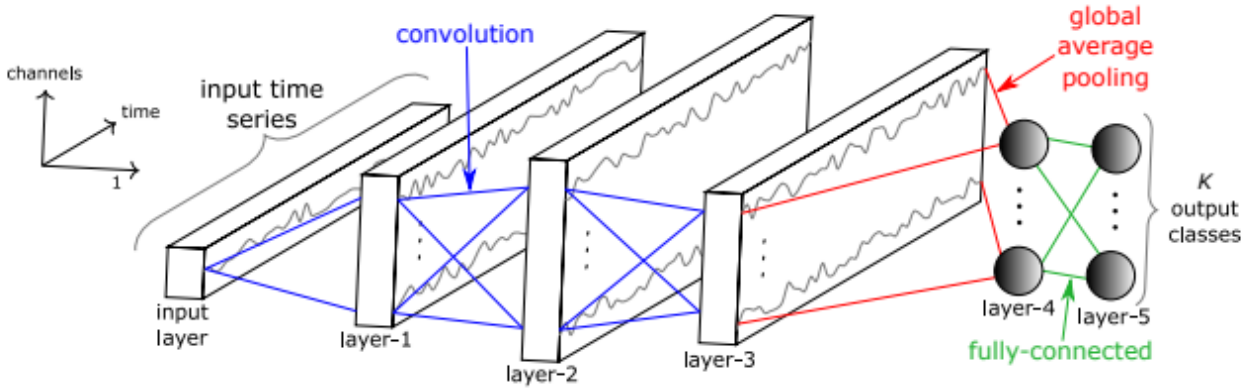
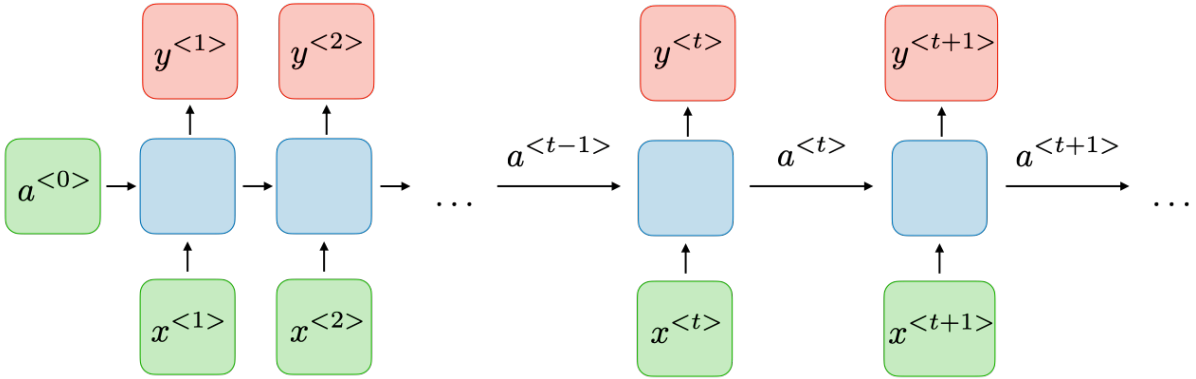


Figure from (Fawaz et al., 2019)

An introduction to RNN (Elman, 1990)



Figures extracted from:
Stanford lecture CS230

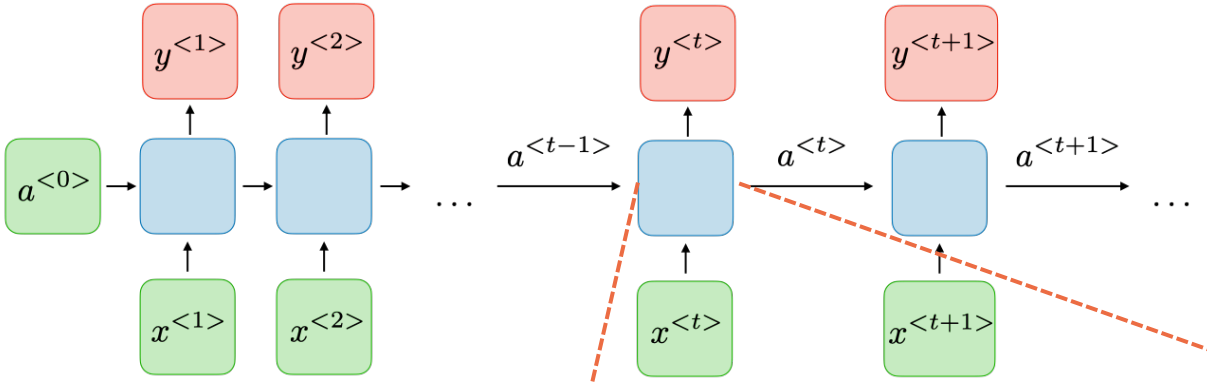
where:

$\{x^{<t>}\}_{t\in\{1;T\}}$ a sequence, $x^{<t>} \in \mathbb{R}^d$

$\{a^{<t>}\}_{t\in\{1;T\}}$ set of hidden states, $a^{<t>} \in \mathbb{R}^l$

$\{y^{<t>}\}_{t\in\{1;T\}}$ set of outputs, $y^{<t>} \in \mathbb{R}$

An introduction to RNN (Elman, 1990)



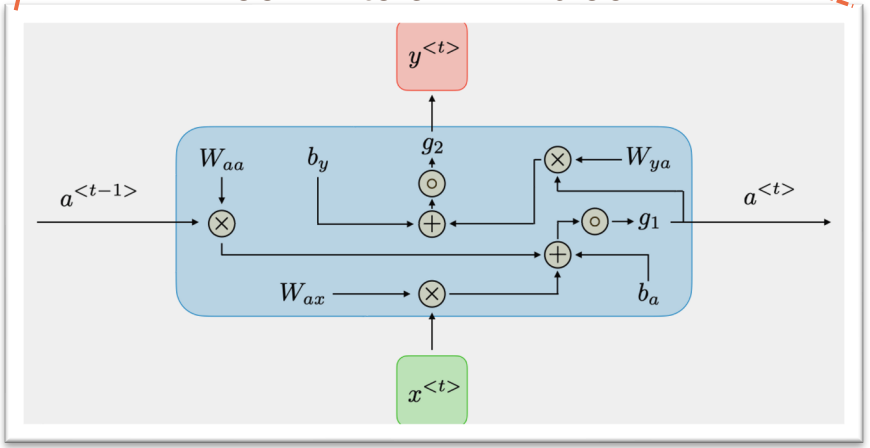
where:

$$a^{<t>} = g_1(a^{<t-1>}, x^{<t>})$$
$$= g_1(W_{aa}a^{<t-1>} + W_{ax}x^{<t>} + b_a)$$

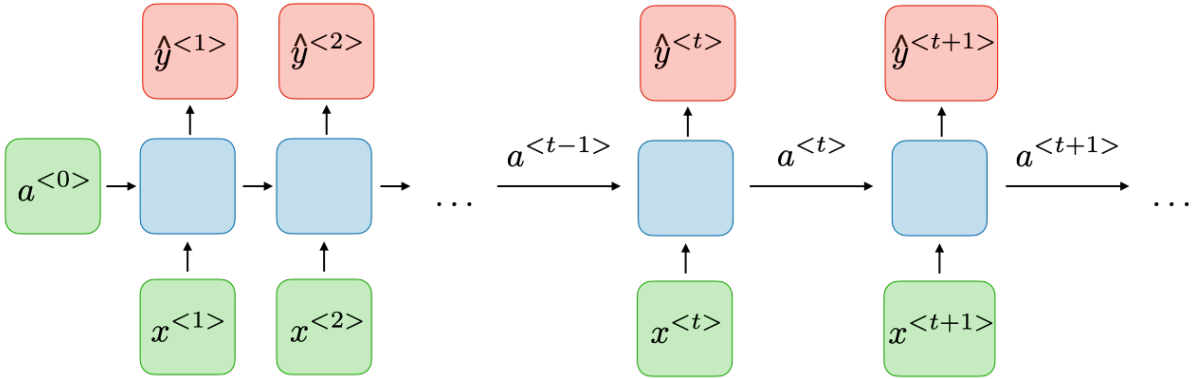
$$y^{<t>} = g_2(W_{ya}a^{<t-1>} + b_y)$$

*shared parameters

Zoom into a RNN block



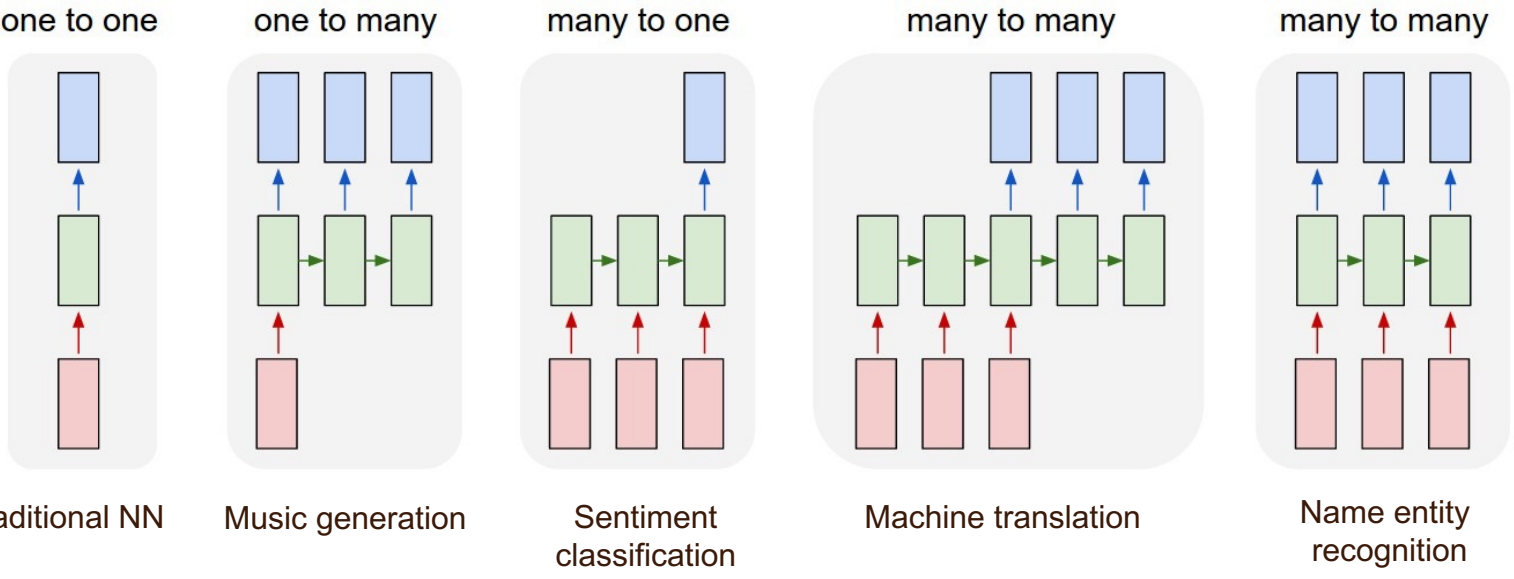
An introduction to RNN (Elman, 1990)



- Loss function in case of classification:
  - at time step t: $\mathcal{L}(\hat{y}^{<t>}, y^{<t>})$
  - total: $\mathcal{L}(\{\hat{y}^{<t>}\}, \{y^{<t>}\}) = \sum_{t=1}^{T} \mathcal{L}(\hat{y}^{<t>}, y^{<t>})$
- The gradient is applied considering the temporality: $\frac{\partial \mathcal{L}}{\partial W} = \sum_{t=1}^{T} \frac{d\mathcal{L}}{dW}\Big|_{(t)}$

The algorithm is called: *backpropagation through time (BPTT)*

Types of RNN

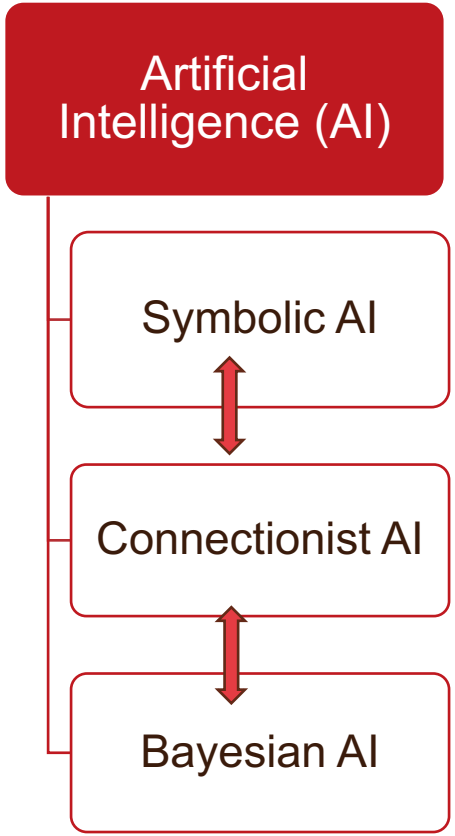

one to one
one to many
many to one
many to many
many to many
Traditional NN
Music generation
Sentiment classification
Machine translation
Name entity recognition

Problems with naive RNN

- Tends to forget old information (distant relationship of unknown length)
- Vanishing gradient problem

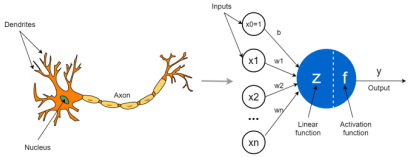
Vocabulary (i)

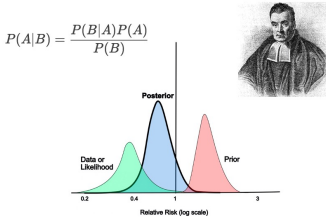
Artificial Intelligence (AI) – main families




```
(defrule 52
 if (site culture is blood)
  (gram organism is neg)
  (morphology organism is rod)
  (burn patient is serious)
 then .4
  (identity organism is pseudomonas))
Rule 52:
 If
  1) THE SITE OF THE CULTURE IS BLOOD
  2) THE GRAM OF THE ORGANISM IS NEG
  3) THE MORPHOLOGY OF THE ORGANISM IS ROD
  4) THE BURN OF THE PATIENT IS SERIOUS
 Then there is weakly suggestive evidence (0.4) that
  1) THE IDENTITY OF THE ORGANISM IS PSEUDOMONAS
```

Extrait de https://en.wikipedia.org/wiki/Mycin





- « **L'intelligence artificielle symbolique** désigne l'ensemble des approches et techniques qui sont fondées sur des représentations « symboliques », c'est-à-dire lisibles par l'homme. ». *Quelques exemples : systèmes experts, logique et raisonnement symbolique, algorithmes de recherche dans les espaces d'états.*

- « **Le connexionnisme** modélise les phénomènes mentaux ou comportementaux, comme des processus émergents de réseaux d'unités simples interconnectées, appelés réseaux de neurones artificiels. » Le connexionnisme est une approche à l'intersection notamment avec les neurosciences.

- « **La statistique bayésienne** est une approche statistique fondée sur l'inférence bayésienne, où la probabilité exprime un degré de croyance en un événement. ». *Quelques exemples : processus gaussien, réseaux bayésiens, modèles de Markov.*

Source: Wikipedia

To go further

Extrait de : Delahaye, J.-P. (2025). IA connexionniste et IA symbolique : l'inévitable association ? Pour la Science, 575(9), 72-77. https://doi-org.docelec.u-bordeaux.fr/10.3917/pls.575.0072.

- Est-il permis d'espérer que l'apprentissage profond seul, un jour, permette aux IA d'acquérir une maîtrise parfaite des opérations entre entiers – quitte à introduire de nouvelles idées dans les méthodes d'apprentissage ou à accroître encore la masse de données mobilisées pour l'entraînement des réseaux de neurones ? Ou l'appel à des outils extérieurs est-il inévitable pour pallier les insuffisances de la méthode connexionniste ? Vers l'IA neuro-symbolique. Avec l'apparition de l'IA Agentique, les chatbots délèguent une partie des tâches à d'autres outils (notamment de calcul).

De l'espoir pour l'IA symbolique

Le problème provient du fait que l'approche symbolique comporte certains avantages dont la contrepartie neuronale est difficile, voire peut-être impossible à trouver. Trois points cristallisent les avantages que l'approche symbolique détient aujourd'hui sur l'approche neuronale.

Tout d'abord, comprendre le fonctionnement des IA est bien plus facile en symbolique qu'en neuronal. En effet, on sait explicitement de quelles connaissances dispose une IA symbolique, ce qui permet d'identifier précisément ce qu'elle exploite pour fonctionner. Les IA à base d'apprentissage profond, à l'inverse, souffrent d'un cruel manque d'explicabilité. Le mécanisme d'un LLM, par exemple, reste essentiellement une boîte noire, dont les milliards de paramètres sont extrêmement difficiles à interpréter – même si, bien sûr, de nombreux travaux de recherche s'y consacrent.

De plus, la fiabilité des résultats donnés par une IA symbolique est souvent très bonne, puisqu'on sait ce qu'elle connaît et comment ses algorithmes traitent cette connaissance. La contrôler et l'ajuster est donc relativement facile. Au contraire, si une IA neuronale se trompe – on peut ici mentionner les fameuses et persistantes hallucinations des IA génératives – il est très délicat de la corriger. Autrement dit, le manque de fiabilité des IA génératives reste un grave problème mal résolu. À l'heure actuelle, personne ne sait, par exemple, comment corriger une IA génératrice d'images pour qu'elle représente correctement un reflet dans un miroir *(voir l'image ci-contre)* ; tout juste peut-on lui montrer des milliers d'images correctes de scènes se reflétant dans un miroir, en espérant que cela conduira le système à corriger ses erreurs.

Un troisième domaine dans lequel l'approche symbolique est bien plus performante que l'approche connexionniste est celui de la performance énergétique : l'électricité nécessaire à la conception des IA symboliques est tout à fait raisonnable, alors que l'entraînement d'un réseau de neurones est extrêmement énergivore. Aujourd'hui, on évalue l'électricité utilisée pour l'entraînement des LLM à environ 1,5 % de la consommation mondiale, et on anticipe que cette proportion doublera d'ici à 2030. Un rapport publié en 2025 par l'Agence internationale de l'énergie indique : « Un centre de calcul dévolu à l'IA consomme autant d'électricité que 100 000 familles, et ceux en construction aujourd'hui en consommeront vingt fois plus. » Certains chercheurs préconisent, sur la base de ce constat, de se passer totalement de l'IA générative à chaque fois que c'est possible.

References

- **[Bengio et al., 2017]** Bengio, Y., Goodfellow, I., & Courville, A. (2017). Deep learning (Vol. 1). Cambridge, MA, USA: MIT press.
- **[LeCun et al., 1998]** LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, *86*(11), 2278-2324.
- **[Ismail Fawaz, et al., 2019]** Ismail Fawaz, H., Forestier, G., Weber, J., Idoumghar, L., & Muller, P. A. (2019). Deep learning for time series classification: a review. *Data mining and knowledge discovery*, *33*(4), 917-963.
- **[Elman, 1990]** Elman, J. L. (1990). Finding structure in time. *Cognitive science*, *14*(2), 179-211.